

Aula 08 — Classificação e Classificadores Gaussianos

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME cap. 7 e §8.1; ISLP cap. 4

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- formular o problema de classificação com a *perda 0–1* e derivar o **classificador de Bayes**;
- descrever a estratégia *plug-in*: estimar $\mathbb{P}(Y = c \mid \mathbf{x})$ e classificar pela maior probabilidade;
- ajustar uma **regressão logística** por máxima verossimilhança e reconhecer sua versão penalizada/multiclasse;
- descrever os *modelos generativos* via Teorema de Bayes: **Bayes ingênuo**, **LDA** e **QDA**;
- entender por que LDA tem fronteira *linear* e QDA *quadrática*;
- contrastar abordagens *discriminativas* e *generativas*.

Em sala

Início do Bloco II. A boa notícia: *quase tudo* da Parte I se transfere — risco, viés-variância, validação cruzada. A mudança é que Y agora é *qualitativo* e a perda natural não é a quadrática, mas a 0–1. Pergunta de abertura: “qual é o melhor classificador possível, se eu conhecesse a distribuição dos dados?”

1 O problema de classificação

Agora Y assume valores num conjunto finito de **classes** $\mathcal{C}$ (ex.: $\mathcal{C} = \{0, 1\}$ no caso binário). Um **classificador** é uma função $g : \mathbb{R}^p \rightarrow \mathcal{C}$. A perda natural é a **perda 0–1**, $L(y, g(\mathbf{x})) = \mathbf{1}\{y \neq g(\mathbf{x})\}$, e o risco é a *probabilidade de erro*:

$$R(g) = \mathbb{E}[\mathbf{1}\{Y \neq g(\mathbf{X})\}] = \mathbb{P}(Y \neq g(\mathbf{X})). \quad (1)$$

Teorema 1.1 (Classificador de Bayes). *Sob a perda 0–1, o classificador que minimiza (1) é*

$$g^*(\mathbf{x}) = \arg \max_{c \in \mathcal{C}} \mathbb{P}(Y = c \mid \mathbf{X} = \mathbf{x}).$$

No caso binário $\mathcal{C} = \{0, 1\}$: $g^*(\mathbf{x}) = 1 \iff \mathbb{P}(Y = 1 \mid \mathbf{x}) \geq \frac{1}{2}$.

Demonstração. (Caso binário, classes c_1, c_2 .) Condicionando em $\mathbf{X}$,

$$R(g) = \mathbb{P}(Y \neq g(\mathbf{X})) = \int [\mathbf{1}\{g(\mathbf{x}) = c_2\} \mathbb{P}(Y = c_1 \mid \mathbf{x}) + \mathbf{1}\{g(\mathbf{x}) = c_1\} \mathbb{P}(Y = c_2 \mid \mathbf{x})] f(\mathbf{x}) d\mathbf{x}.$$

Para cada $\mathbf{x}$, o integrando é minimizado escolhendo $g(\mathbf{x}) = c_1$ quando $\mathbb{P}(Y = c_1 \mid \mathbf{x}) \geq \mathbb{P}(Y = c_2 \mid \mathbf{x})$ e $g(\mathbf{x}) = c_2$ caso contrário — isto é, a classe de maior probabilidade a posteriori. Como cada $\mathbf{x}$ é otimizado pontualmente, o classificador resultante minimiza o risco global. $\square$

Ideia central

O classificador de Bayes g^* é o “oráculo”: nenhum classificador faz melhor. Seu risco R^* é o análogo do *erro irreduzível* da regressão (Aula 01). Mas g^* é *desconhecido*, pois depende de $\mathbb{P}(Y = c \mid \mathbf{x})$. Todo método de classificação é uma tentativa de *aproximá-lo*.

Observação 1.2 (O que se transfere da Parte I). A estimação do risco (1) é feita por *data splitting*/validação cruzada (Aula 03), agora medindo a *proporção de erros* na validação. O balanço viés-variância também vale: $R(g) - R^*$ decompõe-se num termo tipo variância (classe $\mathcal{G}$ grande demais) e num termo tipo viés (classe pequena demais). E, como em regressão, compare *poucos* modelos no teste — faça o *tuning* por CV dentro do treino (a probabilidade de escolher errado cresce com o número de modelos comparados, via Hoeffding + união).

Atenção

A acurácia ($1 - \text{risco } 0-1$) pode **enganar** em dados desbalanceados: com 10 doentes em 1000, o classificador trivial “todos saudáveis” acerta 99% — e é inútil. Precisaremos de outras métricas (matriz de confusão, sensibilidade, precisão) — tema da Aula 09.

2 Classificadores *plug-in*

O Teorema 1.1 sugere uma receita direta, dita *plug-in*:

1. estime $\hat{\mathbb{P}}(Y = c \mid \mathbf{x})$ para cada classe $c \in \mathcal{C}$;
2. classifique $\hat{g}(\mathbf{x}) = \arg \max_{c \in \mathcal{C}} \hat{\mathbb{P}}(Y = c \mid \mathbf{x})$.

Os métodos diferem em *como* estimam $\mathbb{P}(Y = c \mid \mathbf{x})$. Há duas grandes famílias:

Discriminativos

modelam $\mathbb{P}(Y = c \mid \mathbf{x})$ *diretamente* (regressão logística; ou regressão sobre indicadores).

Generativos modelam $f(\mathbf{x} \mid Y = c)$ e $\mathbb{P}(Y = c)$ e invertem com o Teorema de Bayes (Bayes ingênuo, LDA, QDA).

2.1 Via regressão sobre indicadores

Como $\mathbb{P}(Y = c \mid \mathbf{x}) = \mathbb{E}[\mathbf{1}\{Y = c\} \mid \mathbf{x}]$, podemos usar *qualquer* método de regressão (Parte I) sobre $Z = \mathbf{1}\{Y = c\}$. Simples, mas a regressão linear pode dar estimativas fora de $[0, 1]$ (KNN e árvores, por construção, não). Daí o apelo de um modelo que respeite $[0, 1]$: a logística.

3 Regressão logística (discriminativo)

No caso binário $\mathcal{C} = \{0, 1\}$, a regressão logística modela

$$\mathbb{P}(Y = 1 \mid \mathbf{x}) = \frac{e^{\beta_0 + \sum_{i=1}^p \beta_i x_i}}{1 + e^{\beta_0 + \sum_{i=1}^p \beta_i x_i}} = \sigma(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}), \quad (2)$$

onde $\sigma(u) = e^u / (1 + e^u)$ é a função logística. Equivalentemente, o *log-odds* é linear: $\log \frac{\mathbb{P}(Y=1|\mathbf{x})}{\mathbb{P}(Y=0|\mathbf{x})} = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}$. Como em regressão, *não* supomos que isso seja exatamente verdade — esperamos que leve a um bom classificador.

3.1 Estimação por máxima verossimilhança

Dada uma amostra i.i.d., a verossimilhança condicional é

$$L(\beta) = \prod_{k=1}^n \pi(\mathbf{x}_k)^{Y_k} (1 - \pi(\mathbf{x}_k))^{1-Y_k}, \quad \pi(\mathbf{x}_k) = \mathbb{P}(Y_k = 1 \mid \mathbf{x}_k, \beta). \quad (3)$$

Maximiza-se o log da verossimilhança (3). Ao contrário do MQO, não há solução fechada: usam-se métodos numéricos (Newton–Raphson / IRLS). Como na regressão linear, pode-se **penalizar** (ℓ_1/ℓ_2) para reduzir variância (`glmnet`, `family="binomial"`).

3.2 Mais de duas classes

Para $|\mathcal{C}| > 2$: ou ajusta-se uma logística *um-contra-todos* por classe (e normaliza-se), ou usa-se a **regressão logística multinomial** (*softmax*), que garante que as probabilidades estimadas somem 1.

4 Modelos generativos: Teorema de Bayes

Os modelos generativos partem de

$$\mathbb{P}(Y = c \mid \mathbf{x}) = \frac{f(\mathbf{x} \mid Y = c) \mathbb{P}(Y = c)}{\sum_{s \in \mathcal{C}} f(\mathbf{x} \mid Y = s) \mathbb{P}(Y = s)}. \quad (4)$$

Estima-se $\mathbb{P}(Y = c)$ pelas *proporções amostrais* e $f(\mathbf{x} \mid Y = c)$ por algum modelo. As três escolhas abaixo se distinguem pelo modelo de $f(\mathbf{x} \mid Y = c)$.

4.1 Bayes ingênuo

Supõe **independência condicional** das covariáveis dada a classe:

$$f(\mathbf{x} \mid Y = c) = \prod_{j=1}^p f(x_j \mid Y = c). \quad (5)$$

A suposição quase nunca é literalmente verdadeira, mas é *conveniente* (reduz drasticamente o número de parâmetros) e frequentemente gera bons classificadores. Para covariáveis contínuas, modela-se cada $X_j \mid Y = c \sim N(\mu_{j,c}, \sigma_{j,c}^2)$ (parâmetros por classe e por covariável, estimados por MV); para discretas, usa-se multinomial (ótimo em texto — ponte com a Aula extra de NLP).

Atenção

Numericamente, o produto (5) de muitos termos pequenos sofre *underflow* (vira 0). Trabalhe sempre em **escala logarítmica**: $\hat{g}(\mathbf{x}) = \arg \max_c [\sum_j \log \hat{f}(x_j \mid Y = c) + \log \hat{\mathbb{P}}(Y = c)]$.

4.2 Análise discriminante: LDA e QDA

Aqui modela-se o vetor inteiro como **gaussiano multivariado**: $\mathbf{X} \mid Y = c \sim N(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$. A distinção:

- **QDA** (*Quadratic Discriminant Analysis*): cada classe tem sua própria covariância $\boldsymbol{\Sigma}_c$.
- **LDA** (*Linear Discriminant Analysis*): supõe covariância *comum* $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$ para todas as classes.

Proposição 4.1 (LDA tem fronteira linear). Sob $\mathbf{X} | Y = c \sim N(\boldsymbol{\mu}_c, \boldsymbol{\Sigma})$ (covariância comum), o classificador de Bayes (4) equivale a $\hat{g}(\mathbf{x}) = \arg \max_c \delta_c(\mathbf{x})$, com a função discriminante linear

$$\delta_c(\mathbf{x}) = \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c - \frac{1}{2} \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c + \log \mathbb{P}(Y = c).$$

As fronteiras entre classes $\{\mathbf{x} : \delta_c(\mathbf{x}) = \delta_{c'}(\mathbf{x})\}$ são **hiperplanos**.

Demonstração. Classificar por $\arg \max_c \mathbb{P}(Y = c | \mathbf{x})$ equivale a $\arg \max_c \log(f(\mathbf{x} | Y = c) \mathbb{P}(Y = c))$ (o denominador de (4) não depende de c). Com a densidade gaussiana, $\log f(\mathbf{x} | Y = c) = -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) + \text{const.}$ Expandindo o quadrado, o termo $\mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \mathbf{x}$ é igual para todas as classes (pois $\boldsymbol{\Sigma}$ é comum) e pode ser descartado do $\arg \max$. Sobram exatamente os termos lineares em $\mathbf{x}$ de $\delta_c(\mathbf{x})$. A diferença $\delta_c - \delta_{c'}$ é afim em $\mathbf{x}$, logo cada fronteira é um hiperplano. $\square$

Observação 4.2. Em QDA, $\boldsymbol{\Sigma}_c$ varia com c , então o termo $\mathbf{x}^\top \boldsymbol{\Sigma}_c^{-1} \mathbf{x}$ não cancela: as funções discriminantes ficam **quadráticas** em $\mathbf{x}$ e as fronteiras são curvas. Trade-off viés-variância (Aula 01): QDA é mais flexível (menos viés) mas estima muito mais parâmetros (uma matriz de covariância por classe $\Rightarrow$ mais variância). Prefira LDA com poucos dados, QDA com muitos.

5 Discriminativo $\times$ generativo

	Discriminativo (logística)	Generativo (LDA/QDA/NB)
Modela	$\mathbb{P}(Y \mathbf{x})$ direto	$f(\mathbf{x} Y)$ e $\mathbb{P}(Y)$
Suposições	poucas (forma do <i>log-odds</i>)	distribucionais (gaussiana, indep.)
Quando brilha	muitos dados; suposições gaussianas falsas	classes bem separadas; poucos dados; suposições corretas
Bônus	—	gera dados; lida com classes raras

Ideia central

Se as suposições gaussianas valem, LDA/QDA são (quase) ótimos e eficientes com poucos dados. Se não valem, a logística — que assume menos — tende a ser mais robusta com bastante dado. Na dúvida: experimente ambos e compare por validação cruzada.

6 Prática em Python

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.discriminant_analysis import (LinearDiscriminantAnalysis as LDA,
3                                             QuadraticDiscriminantAnalysis as QDA)
4 from sklearn.naive_bayes import GaussianNB
5 from sklearn.pipeline import make_pipeline
6 from sklearn.preprocessing import StandardScaler
7
8 # Logística penalizada (C = 1/lambda); padronizar no pipeline (Aula 07)
9 logit = make_pipeline(StandardScaler(),
10                       LogisticRegression(penalty="l2", C=1.0, max_iter=1000))
11
12 lda, qda, nb = LDA(), QDA(), GaussianNB()
13 for nome, modelo in [("logit", logit), ("LDA", lda), ("QDA", qda), ("NB", nb)]:
14     modelo.fit(X_tr, y_tr)
15     print(nome, "acuracia:", modelo.score(X_te, y_te))
16     # probabilidades plug-in: modelo.predict_proba(X_te)

```

O notebook **Comparação entre classificadores paramétricos** confronta esses métodos; **Aula prática 08** calcula o erro de Bayes numa população conhecida e mede a troca LDA×QDA em $p = 2$ e $p = 10$.

Resumo

- Perda 0–1 (1); o **classificador de Bayes** $g^*(\mathbf{x}) = \arg \max_c \mathbb{P}(Y = c \mid \mathbf{x})$ é ótimo (Teo. 1.1) mas desconhecido.
- *Plug-in*: estimar $\mathbb{P}(Y = c \mid \mathbf{x})$ e tomar o $\arg \max$.
- **Logística** (2): discriminativa, MV, penalizável, multinomial; respeita $[0, 1]$.
- Generativos via Bayes (4): **Bayes ingênuo** (indep. condicional), **LDA** (covariância comum $\rightarrow$ fronteira linear, Prop. 4.1), **QDA** (covariância por classe $\rightarrow$ quadrática).
- Discriminativo (menos suposições, robusto com muito dado) vs. generativo (eficiente sob suposições corretas/poucos dados).
- Cuidado com acurácia em dados desbalanceados $\Rightarrow$ Aula 09.

Leitura recomendada. [AME] Capítulo 7 (risco 0–1, classificador de Bayes, Teorema 9) e §8.1 (*plug-in*, logística, Bayes ingênuo, LDA/QDA); [ISLP] Capítulo 4 (§4.3 logística, §4.4 modelos generativos).